

CPS 200

Session 1
Jan 7, 2026
Kyu Hur

Course Introduction

Course	CPS 200 - Programming I
Semester	Spring 2026
Instructor	Kyu Hur
Instructor Contact Information	khur@japan.lakeland.edu
Class Schedule	Mon, Wed 18:15-19:45
Class Location	510 / MC
Office Hours	Wed, 19:45-20:15

About Me

- Name: Kyu Hur (phonetically Q)
- General:
 - From Korea / United States, grew up in Los Angeles
 - Lived in Tokyo for 7 years

About Me

- Experience:
 - Originally a Chemist, worked in research for 8 years
 - Water chemistry, environmental health, wearables
 - Software Engineer, worked in industry for 5 years
 - Building search systems for a mapping product
- Education:
 - Univ. of Tokyo (MSc. Public Health)
 - Univ. of Illinois (BSc. Chemistry)

Course Introduction

- Take 2-3 minutes to introduce yourself to your neighbors; this class will require a lot of group work and collaboration
- Every session, you should know your neighbor to solve problems together
- Quizzes (starting from the 3rd session) are done separately

Course Introduction – Course Website

- <https://kyuhur2-lakeland.github.io/cps200/>
- Go-to site for past presentations, textbook info, videos, supplementary materials, etc.

Course Introduction – Required Textbooks

- Think Python – 3rd edition (1st half)
 - <https://alldowney.github.io/ThinkPython/index.html>
- Open Data Structures (2nd half)
 - <https://opendatastructures.org/ods-python.pdf>

Course Introduction – Required Supplemental Materials

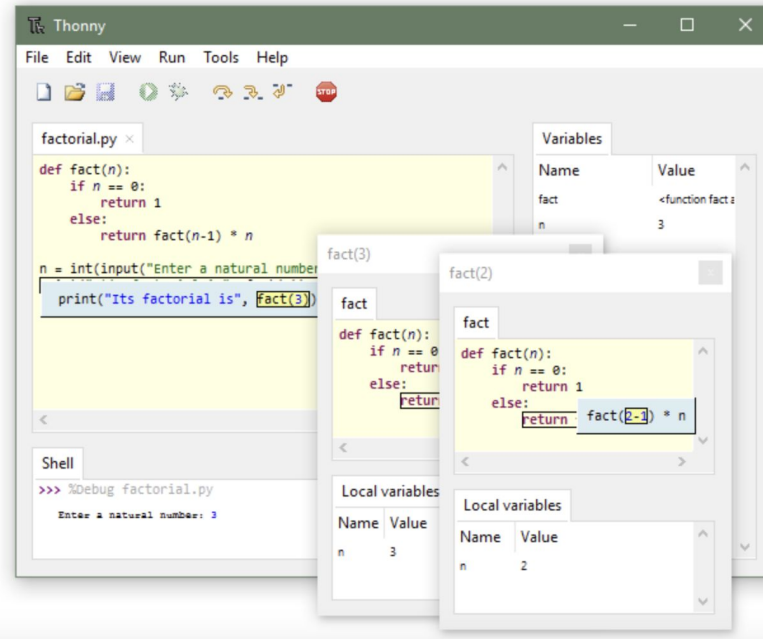
- Thonny (Python IDE)
 - <https://thonny.org/>
- Google Colaboratory (requires a Google account)
 - <https://colab.research.google.com/>

Course Introduction – Required Supplemental Materials

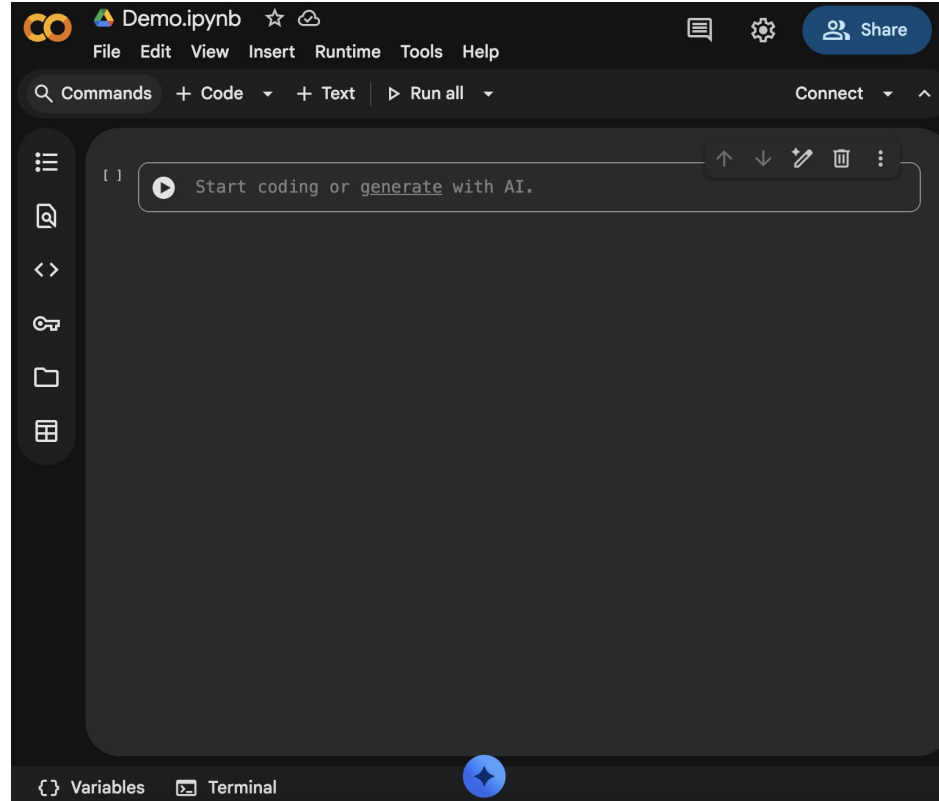
Thonny
Python IDE for beginners



Download version [4.1.7](#) for
[Windows](#) • [Mac](#) • [Linux](#)



Course Introduction – Required Supplemental Materials



Course Introduction

An introduction to computer programming. Emphasis on problem solving techniques, structured programming, top-down program design, control structures, and functions.

Prerequisite:

- MAT 140 - Advanced Algebra

Course Introduction

This is a computer science / applied programming course.

While you will need to learn some basic concepts in math and computer engineering, we will only touch upon what is necessary.

If you are not strong in these subjects, no need to worry. This is mostly a logic based course that borrows from these other fields.

Course Introduction – Learning Outcomes

- Develop the concepts of computer systems and computer programming
- Demonstrate a basic understanding of **binary** numbers and their role in computer programming
- Write clear accurate programs in a high-level language
- Draw flowcharts of programs
- Write program codes to get input data and send output data
- Develop the concept of functions and construct functions
- Write programs that use selection, loops, and functions
- Solve simple problems using fundamental programming techniques and basic problem-solving skills
- Develop programs to implement real-life applications.

Course Introduction – Requirements

Students are expected to bring **a charged, working computer to class every day.**

Computer should have access to **Thonny** and **Google Colab**.

This is very important, as this is an interactive learning course. You will have to write and run code on your machine, so please make sure to always bring your computer to class.

Course Introduction – Policies

Refer to syllabus for general policies on plagiarism.

No use of AI during class.

AI should only be used on your own time to review materials. Use of AI during quizzes, tests, or assignments will result in a 0 for that task and potential further disciplinary action.

Course Introduction – Grades

- A = 93% to 100%
- AB = 88% to 93%
- B = 83% to 88%
- BC = 78% to 83%
- C = 73% to 78%
- CD = 68% to 73%
- D = 60% to 68%
- F = 0% to 60%

Course Introduction – Grade Weighting

- 20% - Midterm
- 20% - Group Project
- 30% - Quizzes (16 quizzes, you can miss 1 quiz)
- 30% - Final

Schedule

Date	Session	Quiz	Topic
Jan 5			Reading Day
Jan 7	1	No	Intro to Programming I
Jan 12			National Holiday
Jan 14	2	No	Think Python: Programming as a way of thinking, Variables and Statements
Jan 19	3	Yes	Think Python: Functions, Functions and Interfaces
Jan 21	4	Yes	Think Python: Conditionals and Recursion, Return Values
Jan 26	5	Yes	Think Python: Iteration and Search, Strings and Regular Expressions
Jan 28	6	Yes	Think Python: Lists, Dictionaries
Feb 2	7	Yes	Think Python: Tuples, Text Analysis and Generation
Feb 4	8	Yes	Think Python: Files and Databases, Classes and Functions

Schedule

Date	Session	Quiz	Topic
Feb 9	9	No	Midterm
Feb 11			National Holiday
Feb 16	10	Yes	Think Python: Classes and Methods, Classes and Objects
Feb 18	11	Yes	Think Python: Inheritance, Python Extras
Feb 23			National Holiday
Feb 25	12	Yes	Open Data Structures: Introduction & Array-Based Lists
Mar 2	13	Yes	Open Data Structures: Linked Lists
Mar 4	14	Yes	Open Data Structures: Hash Tables
Mar 9	15	Yes	Open Data Structures: Binary Trees
Mar 11	16	Yes	Open Data Structures: Heaps

Schedule

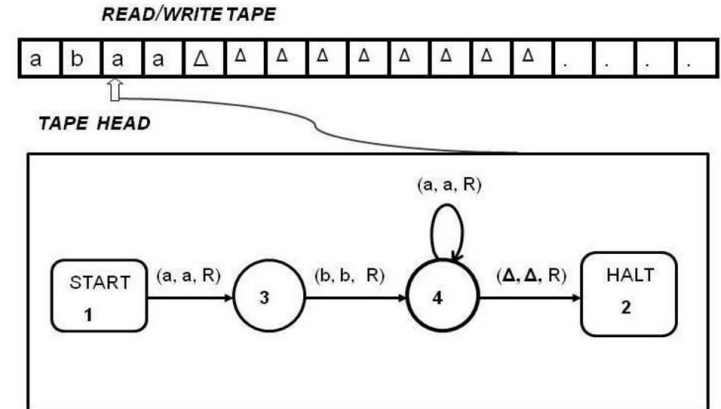
Date	Session	Quiz	Topic
Mar 16	17	Yes	Open Data Structures: Sorting Algorithms
Mar 18	18	Yes	Open Data Structures: Graphs & Group Project
Mar 23	19	Yes	Group Project
Mar 25	20	No	Group Project
Mar 30	21	No	Group Project
Apr 1	22	No	Final Exam (Tentative)

Intro to Programming I – What is a Computer?

- A computer is NOT magical
- It is a clever work of engineering that came about with decades of research during the 1900s
- A computer is a “machine” that can store, send, and process information as **bits** (0 or 1)
- A machine can be theoretically represented by any physical thing

Intro to Programming I – What is a Computer?

- The first theoretical computational machine was the [Turing Machine](#) created by Alan Turing in 1936
 - It describes a machine that *“manipulates symbols on a strip of tape according to a table of rules”*
 - *“Despite the model's simplicity, it is capable of implementing any computer algorithm”*



Intro to Programming I – What is a Computer?

- Modern day machines are simply very complex **Turing Machines**, able to process information much faster and with access to way more data
- With enough material, space and careful engineering, the machines from the 1940s could simulate the same operations as the phone in your pocket
 - Note: This would be impractical as that phone would probably be close to the size of Japan if not larger

Intro to Programming I – What is a Computer?

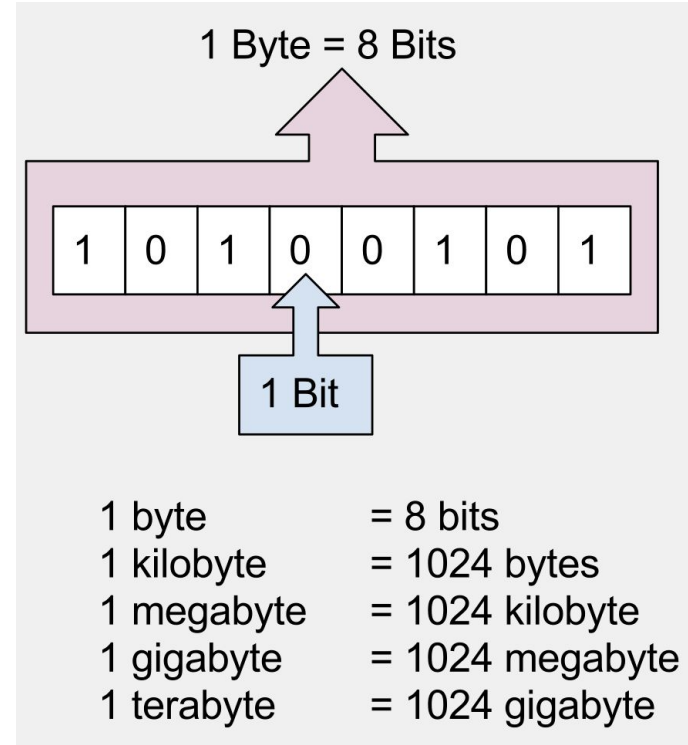
- **Bits (binary digit)** are the most basic representation of data (kind of each section of the tape in the Turing Machine), able to hold only two distinct values
- A **bit** is a representation of the smallest piece of information a computer can have, e.g. a light switch
 - OFF = 0
 - ON = 1
- Computers store, process, and send information to other computers by processing these representations of “0”s and “1”s

Intro to Programming I – What is a Computer?

- A collection of **bits** can be utilized to represent information
- When you have 1 bit, you have 2 representable states:
 - 0 or 1
- When you have 2 bits, you have 4 representable states:
 - 00, 01, 10, or 11
- When you have 3 bits, you have 8 representable states:
 - 000, 001, 010, 011, 100, 101, 110, 111

Intro to Programming I – What is a Computer?

- Data size is usually represented in bytes
- A **byte** is a collection of 8 bits



Intro to Programming I – What is a Computer?

- What is a **bit**?
- Represent the following in bits:
 - 0
 - 1
 - 8
 - 70
 - 255
 - 256
- Can you come up with a formula to go from bits to states? How about states to bits?
- How do you think negative numbers are represented?

Intro to Programming I – What is a Computer?

- Assume **S = # of representable states** and **n = # of bits**
- $S = 2^n$
- $n = \log_2 S$

Intro to Programming I – What is a Computer?

- To go from **bits** to human readable language, humans agreed on a certain dictionary called ASCII to represent bits as letters

Intro to Programming I – What is a Computer?

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Intro to Programming I – What is a Computer?

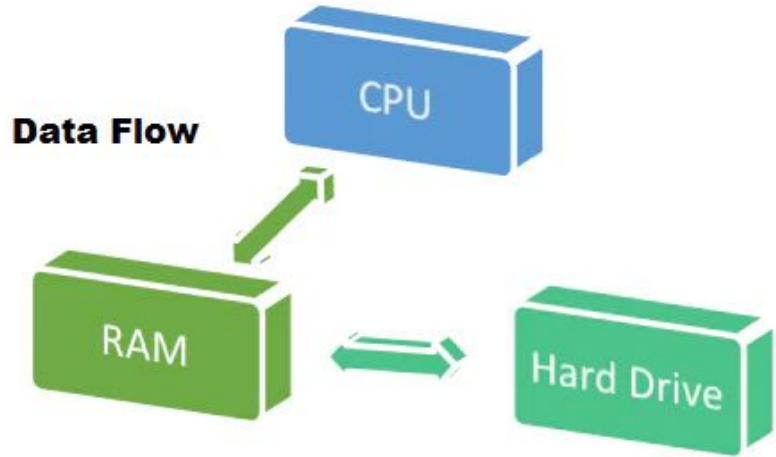
- For example, "Hi" is stored as (middle number is from ASCII table):
 - H -> 72 -> **01001000**
 - i -> 105 -> **01101001**
- Therefore, the collection of 16 bits (or 2 bytes) **01001000 01101001** is understood by the computer to output "Hi"

Intro to Programming I – What is a Computer?

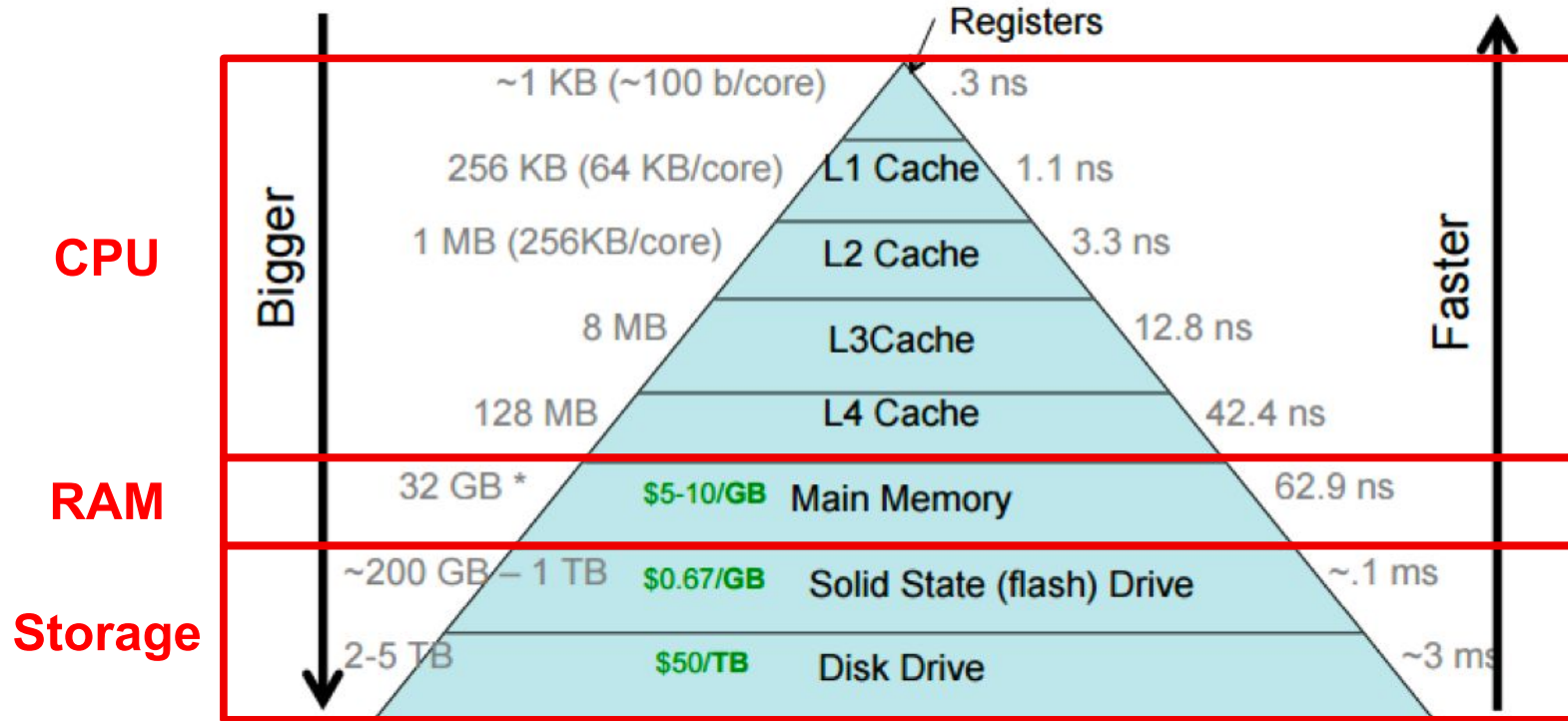
- Can you try writing the following messages in bytes (one by one)?
 - Hello
 - My
 - Name
 - Is
 - <your_name>

Intro to Programming I – What is a Computer?

- A computer is built of multiple components, but you can think of it as three fundamental parts:
 - **Central Processing Unit (CPU)**: the “brain”
 - **Random Access Memory (RAM)**: short-term memory
 - **Storage (SSD/HDD)**: long-term memory



Intro to Programming I – What is a Computer?



1 ns = 1×10^{-9} seconds
1 ms = 1×10^{-3} seconds

Intro to Programming I – What is a Computer?

- Depending on the computer or process, the representation of 0 and 1 is different
- All communication in modern-day computers utilize a complex web of protocols to send, receive, and process information that is provided as 0s and 1s

Intro to Programming I – What is a Computer?

- **Morse code** is a form of “language”
- More precisely a coding or writing system that represents letters, which form a language

A ● -	J ● - - -	S ● ● ●
B - ● ● ●	K - ● -	T -
C - ● - ●	L ● - ● ●	U ● ● -
D - ● ●	M - -	V ● ● ● -
E ●	N - ●	W ● - -
F ● ● - ●	O - - -	X - ● ● -
G - - ●	P ● - - ●	Y - ● - -
H ● ● ● ●	Q - - ● -	Z - - ● ●
I ● ●	R ● - ●	

Intro to Programming I – What is a Computer?

- Bluetooth operates similar to morse code
- Much faster and more complex than morse code but fundamentally the same
- Sends messages from device to device that are a ton of tightly packed 0s and 1s to communicate information



Intro to Programming I – What is a Computer?

- First ever computer, [ENIAC](#), utilized the same concept of 0s and 1s
- It used light bulbs to represent 0s or 1s (off for 0, on for 1)
- It operated like a modern day **function**, giving an output based on a provided input

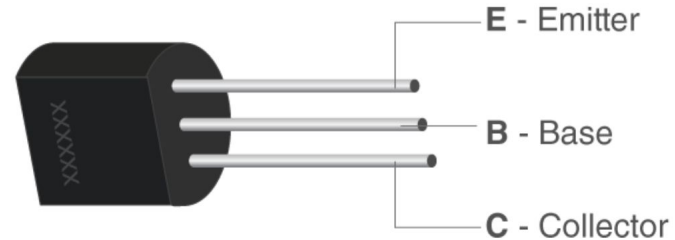
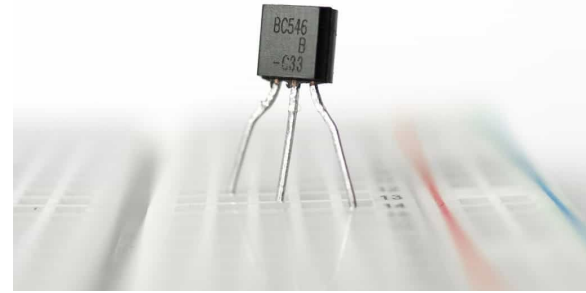


Intro to Programming I – What is a Computer?

- A computer is controlled by a **microprocessor**, which is made up of smaller components called **transistors**.
- What is a **transistor**?

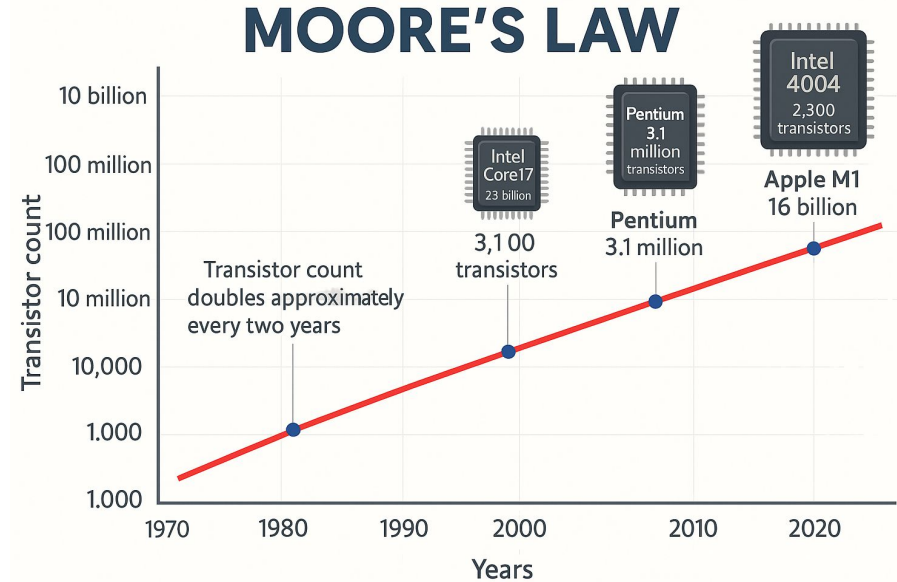
Intro to Programming I – What is a Computer?

- A **transistor** is a tiny electronic switch; which allows electricity to flow through or not
- **Transistors** are the base of all computers today
- More transistors = “stronger” computation



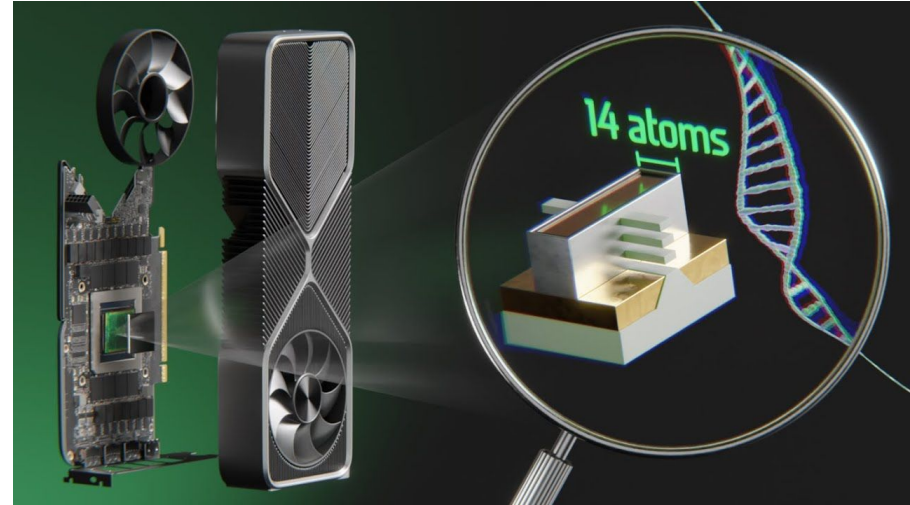
Intro to Programming I – What is a Computer?

- The famous [Moore's Law](#) observed in 1965
- Suggested that the number of transistors in a chip will double every 2 years



Intro to Programming I – What is a Computer?

- Nowadays, **transistors** are so tiny that they are the size of literal atoms
- The smallest transistors are ~14 atoms (2-3 nm) in length



Intro to Programming I – What is a Computer?

- Apple's M5 chip (latest chip) is between 140-150 millimeter²
 - About the size of a fingernail
- Contains ~28 billion **transistors**



Intro to Programming I – What is a Computer?

- Knowing what a transistor does, discuss the following questions:
 - What does it mean to store information?
 - What does it mean to process information?

End